Learning, Adaptation, and Generalization for Autonomous Agents and Robots in Open-World Environments

A dissertation submitted by

Shivam Goel

in partial fulfillment of the requirements
for the degree of

Doctor of Philosophy

in

Computer Science

TUFTS UNIVERSITY

May 2026

©Copyright 2025 by Shivam Goel

Advisors: Jivko Sinapov & Matthias Scheutz

Abstract

Autonomous agents and robots deployed in real-world environments must operate under open-world conditions in which unforeseen changes, novel objects, and unexpected task variations are inevitable. However, most contemporary artificial intelligence and robotics systems are developed under closed-world assumptions where environments, tasks, and dynamics encountered during deployment closely match those observed during training. These assumptions limit the ability of intelligent systems to adapt when prior knowledge becomes insufficient and to generalize robustly across new situations. This dissertation develops methods for enabling autonomous agents and robots to operate effectively in open-world environments through learning, adaptation, and generalization. To this end, it introduces benchmark environments that systematically inject novelty and enable rigorous evaluation of novelty-aware agents. It further develops adaptive planning-learning systems that integrate task and motion planning with reinforcement learning-based recovery mechanisms, allowing agents to detect execution failures and adapt their behavior when previously learned knowledge becomes insufficient. Finally, it demonstrates how robots can acquire generalizable interaction capabilities through object-centric and force-based reinforcement learning policies that enable sustained-contact manipulation across diverse objects and tasks. Together, these contributions provide a unified perspective on building autonomous agents and robots capable of operating in open-world environments. By combining benchmark-driven evaluation, adaptive planning-learning systems, and generalizable embodied interaction policies, this work advances the development of intelligent systems that can robustly operate in complex, dynamic, and evolving real-world settings.

Acknowledgments

I would like to thank...

Contents

Abstract **ii**

Acknowledgments **iii**

1 Introduction **2**

1.1 Thesis Contributions 3

1.2 Thesis Outline 4

2 Background **6**

2.1 Related Work 7

2.1.1 Open-World Sensing and Perception 7

2.1.2 Open-World Planning and Learning 8

2.1.3 Open-World Execution and Recovery 9

2.2 Preliminaries 10

2.2.1 Symbolic Planning 10

2.2.2 Reinforcement Learning 11

2.2.3 Hybrid Planning and Learning 13

2.2.4 Novelty in Open-World Environments 14

3 Benchmark Environments for Open-World Novelty **15**

3.1 Motivation 15

3.2 NovelGridworlds 16

3.2.1 Environment Design 17

3.2.2 Novelty Taxonomy 18

3.2.3 Evaluation Metrics 20
3.2.4 Experimental Results 20
3.2.5 Key Insights 21
3.3 NovelGym 22
3.3.1 Environment Formalism 23
3.3.2 Agent Formalism 24
3.3.3 Novelty as Environment Transformation 25
3.3.4 Architecture and Implementation 26
3.3.5 Evaluation Protocol and Metrics 28
3.3.6 Benchmark Task and Novelties 29
3.3.7 Agent Architectures 30
3.3.8 Experimental Results 32

List of Figures

3.1 Screenshot of the NovelGridworld-Bow-v0 environment. The agent (triangle) faces the crafting table (green). Sticks (yellow) and strings (orange) are distributed in the gridworld. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls. 17

3.2 Representative NovelGridworlds novelty scenarios discussed in the evaluation (fence, axe, and fire wall). Each panel shows how the grid layout and entities change under the corresponding novelty. 19

3.3 Learning performance of the RL agent on NovelGridworld-Bow-v0. The vertical line at 2 million time steps marks the introduction of novelty. Break increase (orange) shows improved reward but reduced success rate. Fire wall (green) causes complete failure. Action remapping (blue) shows partial recovery. 22

3.4 NovelGym environment representation showing a gridworld with various entities. The red box highlights a novelty (axe) introduced in the environment. 23

3.5 Illustration of the agent's sensor representations in NovelGym. (Left) A LiDAR-like representation emitting beams at angular increments. (Right) An image-based local view representation using one-hot vector encodings. 25

3.6 System design of the NovelGym ecosystem. Blue highlights the environment modules and purple highlights the agent modules. The novelty injector (red) transforms the environment based on specified configurations. 26

3.7 Illustration of the NovelGym evaluation protocol and performance over time: success rate as training progresses, with novelty injection dividing pre- and post-novelty regimes. The annotated regions correspond to the metrics in Section 3.3.5 (pre-novelty asymptotic performance, novelty impact, adaptation, and post-novelty performance). 28

3.8 NovelGym environment representation showing a gridworld with various entities. The red box highlights a novelty (axe) introduced in the environment. 30

3.9 Illustration of the pre-novelty environment and three novelty scenarios: fire, fence, and chest. Each transformation modifies specific components of the environment formalism. 31

Chapter 1

Introduction

Autonomous agents and robots are increasingly expected to operate in complex real-world environments such as homes, warehouses, hospitals, and urban spaces. In these settings, agents must interact with diverse objects, perform varied tasks, and respond to changing environmental conditions. Unlike controlled laboratory settings, real-world environments are inherently open, new objects may appear, tasks may change, and interactions may produce outcomes that were not anticipated during system design or training. Developing systems that can operate robustly in such environments remains one of the central challenges in artificial intelligence and robotics.

Most contemporary AI and robotic systems are developed under closed-world assumptions, where the environment, tasks, and dynamics encountered during deployment closely match those observed during training. These assumptions simplify learning and planning but limit the ability of agents to operate reliably when confronted with novelty or unexpected situations. In open-world environments, previously learned models may become incomplete or invalid, action execution may fail due to unforeseen circumstances, and interaction policies may not generalize to new objects or tasks. Addressing these limitations requires systems that can adapt when prior knowledge becomes insufficient and acquire behaviors that generalize across diverse interactions with the environment.

This dissertation investigates how autonomous agents and robots can operate effectively in open-world environments through learning, adaptation, and generalization. The work focuses on three interconnected problems. First, developing environments and benchmarks that systematically introduce novelty and enable rigorous evaluation of open-world agents. Second,

enabling agents to recover from failures during task execution by integrating planning with learning-based adaptation. Third, enabling robots to learn interaction policies that generalize across objects, tasks, and embodiment.

The first part of this dissertation addresses the need for systematic evaluation of open-world agents. We introduce benchmark environments that enable controlled injection of novelty and allow researchers to evaluate how agents respond to unexpected changes in their environment. These environments provide a platform for studying adaptive behavior and for comparing different approaches to open-world autonomy.

The second part of the dissertation focuses on adaptive planning-learning systems that allow agents to recover from execution failures. In open-world environments, plans may fail because of incomplete models or unexpected environmental changes. This work develops mechanisms that integrate task and motion planning with reinforcement learning-based recovery strategies, allowing agents to detect failures and adapt their behavior when previously learned knowledge becomes insufficient.

The third part of the dissertation investigates how robots can acquire generalizable interaction capabilities. Real-world robotic tasks often require sustained contact with objects and interactions that differ from those encountered during training. This work explores object-centric and force-based reinforcement learning approaches that enable robots to learn manipulation policies capable of generalizing across objects, tasks and particular robotic embodiments.

Taken together, the chapters of this dissertation develop a unified perspective on building autonomous agents and robots capable of operating in open-world environments. By combining benchmark-driven evaluation, adaptive planning-learning systems, and generalizable embodied interaction policies, this work advances the development of intelligent systems that can robustly operate in the real world.

1.1 Thesis Contributions

This dissertation develops methods for enabling autonomous agents and robots to operate effectively in open-world environments through learning, adaptation, and generalization. The contributions address how such systems can be evaluated under novelty, how they can adapt

when execution failures occur, and how robots can acquire interaction policies that generalize across objects, tasks, and robotic embodiment.

The main contributions of this dissertation are summarized as follows:

- It introduces benchmark environments and evaluation methodologies that systematically introduce novelty into agent environments, enabling rigorous study of autonomous systems operating under open-world conditions. These environments provide mechanisms for controlled novelty introduction and support evaluation of how agents detect, respond to, and adapt to unexpected changes in their environment.

- It develops adaptive planning–learning systems and architectural frameworks that enable agents to recover from execution failures in changing environments. By integrating planning mechanisms with learning-based adaptation, these systems allow agents to detect when previously learned models become insufficient and modify their behavior to continue task execution despite unexpected changes.

- It introduces object-centric reinforcement learning approaches for robotic interaction that learn manipulation policies in force space. These methods enable robots to acquire interaction strategies that generalize across objects, tasks, and robot embodiments, supporting manipulation behaviors involving sustained contact and diverse physical interaction dynamics.

1.2 Thesis Outline

The remainder of this dissertation is organized as follows.

Chapter 2 presents background and related work relevant to open-world autonomy. It discusses the limitations of closed-world assumptions in contemporary artificial intelligence and robotics systems and reviews prior work in task and motion planning, reinforcement learning, novelty-aware agents, and robotic manipulation. The chapter establishes the conceptual foundations used throughout the dissertation.

Chapter 3 introduces benchmark environments designed to systematically introduce novelty and evaluate agent behavior under open-world conditions. These environments provide mecha-

nisms for controlled novelty injection and enable rigorous evaluation of how agents detect and respond to unexpected changes in their environment.

Chapter 4 develops adaptive planning–learning systems that allow agents to recover from execution failures during task execution. The chapter presents approaches that integrate planning mechanisms with learning-based adaptation, enabling agents to detect when previously learned models become insufficient and modify their behavior in response to unexpected situations.

Chapter 5 investigates how robots can acquire interaction policies that generalize across objects and tasks. It presents object-centric and force-based reinforcement learning approaches for manipulation tasks involving sustained contact and complex object dynamics, demonstrating how these methods enable robots to learn interaction strategies that transfer across diverse environments and manipulation scenarios.

Chapter 6 concludes the dissertation by summarizing the main findings and discussing future directions for research on autonomous agents and robots operating in open-world environments.

Chapter 2

Background

This chapter provides the background necessary for the remainder of the dissertation. It is organized in two parts. The first part surveys prior work on open-world autonomy, situating this thesis within the broader landscape of AI and robotics research. The second part introduces the formal frameworks that underpin the technical contributions of the dissertation.

The dissertation is grounded in the classical agent–environment paradigm, in which an agent perceives its surroundings, reasons and plans over possible actions, and executes those actions to achieve its goals. This perceive–plan–execute loop, and its natural extension to include adaptation and recovery, provides a unifying lens through which to examine the challenges of open-world autonomy. In closed-world settings, each stage of this loop has been studied extensively and with considerable success. However, when agents are deployed in open-world environments, the assumptions underlying each stage are violated: new objects may appear, dynamics may change, and actions may fail in unforeseen ways. Critically, novelty at any stage of the loop can propagate failures across the others, making open-world autonomy a systemic challenge rather than an isolated one.

Section 2.1 reviews prior work along each stage of the agent loop under open-world conditions – sensing, planning and learning, and execution and recovery—and identifies the gaps that motivate the contributions of this dissertation. Section 2.2 then introduces the formal models used throughout: symbolic planning (Section 2.2.1), reinforcement learning (Section 2.2.2), hybrid planning and learning systems (Section 2.2.3), and a formalization of novelty relative to an agent's knowledge and models (Section 2.2.4).

2.1 Related Work

This section reviews prior work relevant to each stage of the perceive–plan–execute loop under open-world conditions. Rather than providing exhaustive coverage of any single area, the discussion is organized around three pillars of open-world autonomy—sensing and perception, planning and learning, and execution and recovery—and aims to identify the key challenges and limitations that motivate the contributions of this dissertation.

2.1.1 Open-World Sensing and Perception

In closed-world settings, perception systems are designed to recognize a fixed set of categories, objects, or states defined at training time. Open-world perception, by contrast, requires the ability to detect and reason about entities and situations that were not anticipated during system design. Early work on open set recognition [34, 4] extended standard classifiers to reject inputs that do not belong to any known category, typically through confidence thresholding or distribution boundary modeling. Out-of-distribution detection methods [20] further developed principled approaches for identifying inputs that fall outside the training distribution.

More recently, vision-language models (VLMs) have transformed the landscape of open-world perception. CLIP [32] demonstrated that joint vision-language pretraining enables zero-shot recognition of arbitrary categories described in natural language, fundamentally relaxing the closed-vocabulary constraint. This capability has been extended to object detection through open-vocabulary detectors such as ViLD [15] and OWL-ViT [28], which can localize objects described by free-form text queries without category-specific training. Foundation models for segmentation, such as the Segment Anything Model [24], further demonstrate that general-purpose visual representations can support open-ended perception tasks.

Despite these advances, a critical gap remains between perceptual novelty detection and the needs of an acting agent. Recognizing that an object is unfamiliar or that a scene has changed is necessary but not sufficient: the agent must also determine whether the novelty is functionally relevant to its current task and how it should modify its behavior in response. Current perception systems largely operate as passive classifiers and do not reason about the implications of detected novelty for downstream planning and execution.

2.1.2 Open-World Planning and Learning

Planning and learning provide the reasoning and decision-making capabilities that allow agents to select actions in pursuit of goals. Classical symbolic planning, formalized through STRIPS [8] and PDDL [27], enables structured long-horizon reasoning by searching over sequences of operators with well-defined preconditions and effects. Modern planners such as Fast Downward [19] achieve strong performance across diverse domains through domain-independent heuristics. However, symbolic planning assumes that the domain model is complete and correct; when the model is inaccurate or missing operators, generated plans may fail during execution.

Reinforcement learning offers a complementary approach, enabling agents to learn effective behaviors through interaction without requiring explicit models. Deep RL algorithms, including DQN [29], PPO [38], and SAC [17], have demonstrated success in complex control tasks. Model-based approaches, including world models such as Dreamer [18] and planning-based methods such as MuZero [37], learn environment dynamics and use them for decision-making. These methods reduce the need for hand-specified models but typically assume that the learned dynamics remain stationary.

A recent and rapidly growing line of work leverages large pretrained models for robotic planning and decision-making. Vision-language-action (VLA) models such as RT-2 [44] directly map visual observations and language instructions to robot actions, drawing on the broad world knowledge encoded in large language and vision models. SayCan [1] and related approaches [22] use large language models as high-level planners that decompose natural language instructions into sequences of learned skills. Generalist robot policies such as Octo [40] and π_0 [5] aim to learn cross-embodiment policies from large-scale multi-robot datasets. While these approaches demonstrate impressive breadth, they face challenges related to physical grounding, hallucination of infeasible actions, and robustness to environmental conditions that diverge from their training data. In open-world settings, where the nature of the change may be qualitatively different from anything in the training distribution, the reliability of these systems remains an open question.

2.1.3 Open-World Execution and Recovery

Even when perception and planning succeed, action execution in open-world environments may fail due to unexpected dynamics, novel object properties, or environmental changes. Recovery from such failures requires mechanisms that go beyond standard error handling: the agent must identify why the failure occurred and adapt its behavior accordingly.

Classical approaches to execution-time adaptation include plan monitoring and repair [9, 10], which detect deviations from expected execution outcomes and revise plans within the existing domain model. However, plan repair assumes that the domain model remains valid and that a correct plan exists within the current model, which may not hold when qualitative novelty has occurred. Continual learning methods [25, 33] address the problem of acquiring new capabilities without forgetting previous ones, but most assume known task boundaries and distributions. Sim-to-real transfer techniques, including domain randomization [42] and system identification [31], improve policy robustness by training across varied conditions, but the variations are typically parameterized in advance and may not cover the kinds of qualitative changes encountered in open-world settings.

More recently, work within the DARPA SAIL-ON program [39] has specifically targeted the problem of agents that must detect, characterize, and accommodate novelty in their environments. This program motivated research on novelty as an agent-relative concept: what constitutes novelty depends on the agent's prior knowledge and capabilities, not solely on properties of the environment [6, 26]. Several approaches have explored how agents can adapt their policies or models in response to detected novelty [2, 13], though systematic methods for efficient recovery from qualitatively new failure modes remain limited. Some efforts have sought to address multiple stages of the open-world agent loop in an integrated manner. World models [16, 18] offer a potential unifying framework by learning environment dynamics that can serve both perception and planning, though they typically do not handle discrete, qualitative changes in the environment. End-to-end foundation models for robotics [44, 40] implicitly combine perception, reasoning, and action selection in a single model, but their behavior under genuine novelty is not well understood. Task and motion planning systems [11, 23] integrate symbolic and geometric reasoning but inherit the brittle model assumptions of classical planning. De-

spite growing interest in open-world autonomy, the field remains largely fragmented: methods that address perception under novelty, planning under incomplete models, and recovery from execution failures are typically developed in isolation from one another.

The preceding discussion has examined open-world autonomy through the lens of the perceive–plan–execute loop, identifying challenges at each stage: sensing under novelty, planning and learning with incomplete models, and execution and recovery when prior knowledge becomes insufficient. Drawing primarily from the challenges identified in planning and learning (Section 2.1.2) and execution and recovery (Section 2.1.3), this dissertation makes progress on three interconnected challenges. First, the lack of standardized environments for systematically evaluating agents under controlled novelty limits rigorous comparison of approaches and slows progress in open-world AI. Second, existing hybrid planning-learning systems do not adequately address novelty handling: they lack mechanisms for detecting execution failures caused by qualitative environmental changes, learning recovery behaviors, and reusing acquired knowledge across subsequent encounters with novelty. Third, robotic skill learning methods remain limited in their ability to produce interaction policies that generalize across objects and platforms, particularly for contact-rich manipulation tasks.

These three challenges correspond to the core contributions of this dissertation. Chapter 3 introduces benchmark environments for evaluating open-world agents under controlled novelty injection. Chapter 4 develops hybrid planning-learning architectures that enable agents to adapt and recover when previously learned models become insufficient. Chapter 5 presents object-centric, force-based reinforcement learning methods for robotic manipulation that generalize across objects and robot embodiments. Together, these contributions advance the goal of building autonomous agents and robots that can operate robustly in open-world environments where novelty is not the exception but the norm.

2.2 Preliminaries

2.2.1 Symbolic Planning

Symbolic planning models perform sequential decision-making using an explicit representation of states, actions, and goals through symbols. The environment is described using a first-

order language consisting of predicate symbols and function symbols. Predicate symbols define Boolean properties of objects, while function symbols define numeric quantities.

A fluent is a state variable whose value may change over time. Boolean fluents are obtained by grounding predicate symbols with objects in the domain, and numeric fluents are obtained from function symbols. Let $\mathcal{F}$ denote the set of all fluents. A symbolic state $\tilde{s} \in \tilde{\mathcal{S}}$ is defined as an assignment of values to the fluents in $\mathcal{F}$, assigning each Boolean fluent a value in $\{0, 1\}$ and each numeric fluent a value in $\mathbb{R}$. In propositional domains, a symbolic state can be equivalently represented as the set of grounded predicates that are true, with all others assumed false under the closed-world assumption.

The symbolic model is defined by a finite set of operators $\mathcal{O}$. Each operator $o \in \mathcal{O}$ is defined as $o = \langle \psi_o, \omega_o \rangle$, where ψ_o is a logical formula over fluents specifying when the operator is applicable, and ω_o specifies how the state changes after execution. An operator o is applicable in a state $\tilde{s}$ if $\tilde{s} \models \psi_o$. Effects modify Boolean fluents through add and delete operations and update numeric fluents through assignments or arithmetic expressions. Operators are typically defined as parameterized schemas and instantiated over objects to produce grounded operators.

A planning task is defined as $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$, where $\tilde{s}_0 \in \tilde{\mathcal{S}}$ is the initial state and g is a logical condition over fluents. A plan is a finite sequence of operators $\rho = [o_1, \ldots, o_n]$. Executing a plan from the initial state produces a sequence of symbolic states obtained by sequentially applying applicable operators. A plan is valid if execution from $\tilde{s}_0$ results in a state that satisfies g. Classical symbolic planning assumes deterministic transitions, where the effects of an operator uniquely determine the resulting state.

Planning domains and problems are commonly specified using the Planning Domain Definition Language (PDDL), which provides a standard representation for predicates, functions, operators, and planning tasks.

2.2.2 Reinforcement Learning

Reinforcement learning models sequential decision-making in which an agent learns to select actions through interaction with an environment so as to maximize cumulative reward. At each time step, the agent observes a state, selects an action, receives a reward signal, and transitions to a new state.

Reinforcement learning problems are modeled as a Markov Decision Process (MDP), defined as $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(s' \mid s, a)$ defines the probability of transitioning to state s' after taking action a in state s, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor.

At time step t, the agent is in state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives reward $r_{t+1} = R(s_t, a_t)$, and transitions to a new state s_{t+1} according to $P(\cdot \mid s_t, a_t)$. The Markov property assumes that the transition and reward depend only on the current state and action.

A policy $\pi(a \mid s)$ defines the behavior of the agent by specifying a probability distribution over actions given a state. The objective of reinforcement learning is to find a policy that maximizes the expected cumulative reward.

The return from time step t is defined as $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$. The state-value function is defined as $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$, and the action-value function is defined as $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$.

These value functions satisfy recursive relationships known as the Bellman expectation equations. The state-value function satisfies

$$V^\pi(s) = \mathbb{E}_\pi \left[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s \right],$$

and the action-value function satisfies

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[r_{t+1} + \gamma \mathbb{E}_{a' \sim \pi(\cdot \mid s_{t+1})} Q^\pi(s_{t+1}, a') \mid s_t = s, a_t = a \right].$$

In discrete settings, the state and action spaces are finite, i.e., $|\mathcal{S}| < \infty$ and $|\mathcal{A}| < \infty$. In this case, value functions such as $V^\pi(s)$ and $Q^\pi(s, a)$ can be represented explicitly in tabular form, and reinforcement learning algorithms can directly estimate these quantities for each state or state-action pair.

In continuous settings, the state space $\mathcal{S} \subseteq \mathbb{R}^n$ and/or the action space $\mathcal{A} \subseteq \mathbb{R}^m$ may be continuous. In such cases, value functions and policies cannot be represented exactly and are instead approximated using parameterized function approximators, typically implemented using neural networks. A policy is represented as $\pi_\theta(a \mid s)$, where θ denotes the parameters

of the policy, and value functions are represented as $V_\phi(s)$ and $Q_\phi(s, a)$, where ϕ denotes the parameters of the function approximator.

Learning in continuous settings involves optimizing the parameters θ and ϕ to maximize expected return. These formulations are particularly important in robotics and control tasks, where states and actions are naturally continuous and policies must operate over high-dimensional spaces.

2.2.3 Hybrid Planning and Learning

Symbolic planning and reinforcement learning provide complementary approaches to sequential decision-making. Symbolic planning enables structured reasoning over high-level representations but assumes accurate and complete models of the environment. Reinforcement learning enables agents to learn behaviors through interaction but typically operates over lower-level or subsymbolic state representations and may lack explicit structure. In many settings, reinforcement learning can also be viewed hierarchically, where high-level symbolic operators correspond to abstract actions, and their execution is realized through lower-level policies or planners. Hybrid planning and learning systems aim to combine these paradigms by integrating symbolic reasoning with learned or executed behaviors.

In hybrid systems, decision-making is defined over two representations. The symbolic model is represented by a planning task $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$,and the execution is formalized by a Markov decision process $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$. The symbolic model operates over discrete representations, whereas the execution model operates over environment states that may be continuous or stochastic. An abstraction function $\alpha : \mathcal{S} \to \tilde{\mathcal{S}}$ maps environment states to symbolic states.

Operators in the symbolic model correspond to executable behaviors in the environment. Each operator $o \in \mathcal{O}$ is associated with a policy $\pi_o(a \mid s)$ that defines its execution in the environment. A symbolic plan $\rho = [o_1, \ldots, o_n]$ therefore induces a sequence of policies executed in the environment.

A hybrid planning and learning system is defined as $\mathcal{H} = \langle \mathcal{T}, M, \alpha \rangle$, which formalizes the separation between symbolic decision-making over abstract representations and execution in the environment.

2.2.4 Novelty in Open-World Environments

Novelty is defined relative to an agent's knowledge. An element of the environment is considered novel if it cannot be represented or inferred using the agent's current models of the world.

Let the agent's knowledge be defined by a symbolic model $\mathcal{T} = \langle \mathcal{F}, \mathcal{O}, \tilde{s}_0, g \rangle$ and an execution model $M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$. The symbolic model specifies structured knowledge over abstract states and operators, while the execution model specifies the dynamics of interaction with the environment.

A novelty corresponds to the introduction of elements that are not contained in the agent's current knowledge. These may include new fluents, operators, states, actions, or changes in transition dynamics. Formally, a novelty induces an extension of the agent's model, such as $\mathcal{F}'$, $\mathcal{O}'$, $\mathcal{S}'$, or $\mathcal{A}'$, where at least one of the following holds:

$$\mathcal{F}' \cap \mathcal{F} = \emptyset, \quad \mathcal{O}' \cap \mathcal{O} = \emptyset, \quad \mathcal{S}' \not\subseteq \mathcal{S}, \quad \mathcal{A}' \not\subseteq \mathcal{A}.$$

In hybrid planning and learning systems, novelty may manifest as a discrepancy between symbolic reasoning and execution. In particular, an operator $o \in \mathcal{O}$ may be applicable in a symbolic state $\tilde{s}$, i.e., $\tilde{s} \models \psi_o$, but its execution in a corresponding environment state may fail to produce the intended effects. Such discrepancies arise when the agent's models are incomplete with respect to the true environment.

This formulation captures novelty as a deviation between the agent's current knowledge and the environment, and provides a basis for reasoning about adaptation in open-world settings.

Chapter 3

Benchmark Environments for Open-World Novelty

3.1 Motivation

As discussed in Chapter 1, developing agents that can operate in open-world environments requires the ability to detect, respond to, and recover from novelty. Evaluating whether an agent possesses these capabilities, however, is far from straightforward. The unbounded nature of novelty means that agents cannot be tested against a fixed, closed set of scenarios. Instead, evaluation environments must support the systematic introduction of diverse novelties while accommodating agents with fundamentally different architectures and representations.

Existing environments for training and evaluating AI agents have largely been designed under closed-world assumptions. Standard reinforcement learning benchmarks such as OpenAI Gym [7] and Atari [3] provide well-defined tasks with fixed dynamics, offering no mechanism for injecting environmental changes after training. Planning benchmarks based on PDDL [27] similarly assume that the domain specification is complete and correct. While some environments support procedural generation or domain randomization, these techniques focus on generalization through diversity rather than on evaluating an agent's response to specific, structured changes in its environment.

Several challenges arise when attempting to evaluate novelty-aware agents. First, the notion

of novelty is inherently agent-relative: what constitutes a novelty for one agent may not be novel for another, depending on each agent's perceptual capabilities, knowledge representation, and prior experience. Second, comparing agents across paradigms is difficult when environments are designed with a particular approach in mind. A planning agent requires symbolic state descriptions and operator definitions, whereas a reinforcement learning agent requires numeric observations and reward signals. An evaluation environment must support both kinds of interfaces without favoring one over the other. Third, the metrics commonly used to evaluate agents, such as cumulative reward or task success rate, are insufficient for characterizing how an agent responds to novelty. Metrics must capture not only whether the agent eventually succeeds, but also how quickly it detects the novelty, how severely its performance degrades, and how efficiently it adapts.

This chapter presents two benchmark environments that address these challenges. The first, NovelGridworlds [12], introduces a benchmark with a rich taxonomy of novelties across multiple tasks, together with evaluation metrics for both planning and reinforcement learning agents. The second, NovelGym [14], extends and generalizes this work into a flexible ecosystem with a formal framework for environment transformations, a modular architecture for developing hybrid agent systems, and adaptation-focused evaluation metrics. Together, these environments provide the infrastructure upon which the adaptive methods presented in subsequent chapters are developed and evaluated, and they contribute to the broader research community by offering standardized benchmarks for open-world novelty. Source codes and documentation for both environments are publicly available, and they have been designed to be extensible so that future researchers can build upon them to explore new types of novelty and adaptation strategies.

3.2 NovelGridworlds

NovelGridworlds[1] is an OpenAI Gym-based benchmark environment designed for developing and evaluating AI agents that can detect and adapt to novelties in open worlds. The environment provides four crafting tasks of increasing complexity, a taxonomy of twelve novelties

[1]The open-source NovelGridworlds repository is available at `https://github.com/gtatiya/gym-novel-gridworlds`. Additional implementation details, including environment wrappers, observation space configurations, and trajectory recording utilities, are described in Goel et al. [12].

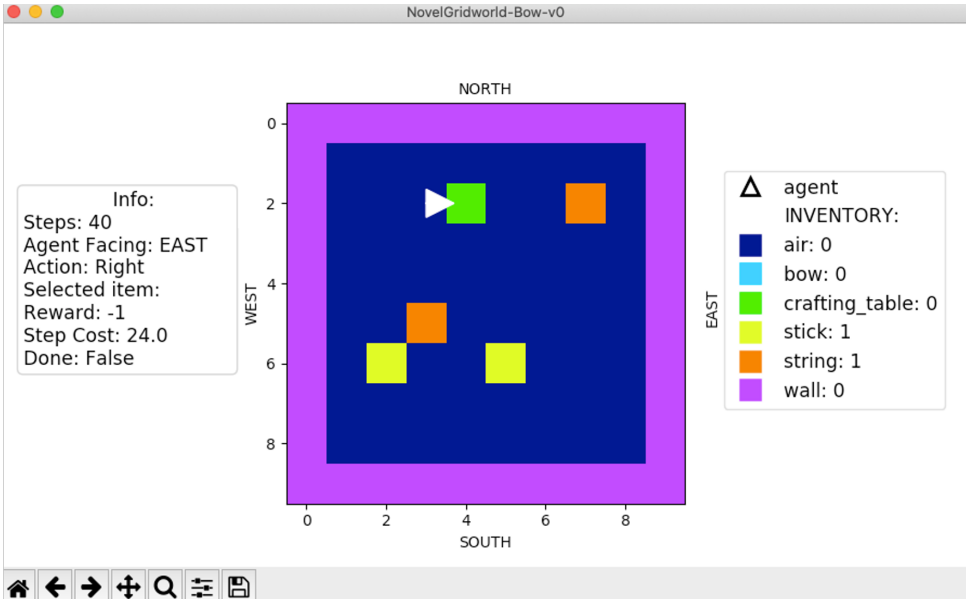


Figure 3.1: Screenshot of the NovelGridworld-Bow-v0 environment. The agent (triangle) faces the crafting table (green). Sticks (yellow) and strings (orange) are distributed in the gridworld. Blue spaces represent air blocks through which the agent can navigate, and purple blocks represent walls.

per task, cross-platform support through a socket interface, and evaluation metrics for both planning and reinforcement learning agents.

3.2.1 Environment Design

Each NovelGridworlds environment is a discrete gridworld of configurable size. The arena is enclosed by unbreakable walls and contains a crafting table, an agent, free space (air blocks), and task-specific objects. At each time step, the agent selects from a discrete set of actions and maintains an inventory of collected items. The environment returns an action cost for each action taken, enabling plan-cost-based evaluation. All environments share a common set of primitive actions: `move-forward`, `turn-left`, `turn-right`, and `break`. Task-specific actions extend this set to support crafting and resource extraction.

Four tasks are implemented, each requiring the agent to craft a target item by collecting resources and applying recipes in the correct sequence:

1. **NovelGridworld-Bow-v0**: The agent collects three sticks and three strings from the environment and crafts a bow at the crafting table.

2. **NovelGridworld-Bow-v1**: An advanced version of the previous task in which the agent must collect raw materials (tree logs and wool) and craft intermediary items (planks, sticks, and strings) before crafting the bow.

3. **NovelGridworld-Pogostick-v0**: The agent collects sticks, planks, and rubber (extracted using a tree tap) to craft a pogostick.

4. **NovelGridworld-Pogostick-v1**: An advanced version in which the agent must craft all intermediary items, including the tree tap itself, from raw tree logs before crafting the pogostick.

The environment communicates with agents through a socket interface, enabling agents implemented in different programming languages to interact with the same environment. This design supports both planning agents implemented in Java-based architectures and reinforcement learning agents implemented in Python.

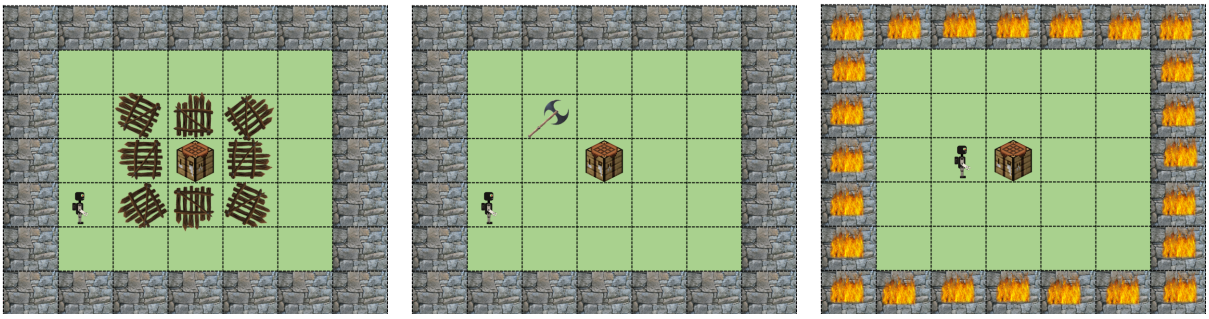
3.2.2 Novelty Taxonomy

A central contribution of NovelGridworlds is a structured taxonomy for classifying novelties along two dimensions. The first dimension categorizes novelties by which component of the agent's model they affect. The second dimension characterizes the impact of the novelty on task performance.

Novelty classes. Using the formalism from Sections 2.2.1 and 2.2.2, novelties are classified into three classes based on the component they alter:

- **Object novelty**: A previously unknown entity appears in the environment, altering the observation space. The resulting state $s' \in \mathcal{S}$ does not come from the distribution of states the agent has previously encountered.

- **Attribute novelty**: The properties of existing entities change, modifying the transition dynamics. Formally, the transition function changes such that for some known state-action pair (s, a), the resulting state $P(\cdot \mid s, a)$ differs from what the agent has previously observed.



(a) Fence (object, detrimental) (b) Axe (object/attribute, beneficial) (c) Fire wall (object/attribute, detrimental)

Figure 3.2: Representative NovelGridworlds novelty scenarios discussed in the evaluation (fence, axe, and fire wall). Each panel shows how the grid layout and entities change under the corresponding novelty.

- **Action novelty**: The behavior of actions changes. An action that previously produced a known effect now produces a different outcome, or new actions become available.

Novelty types. Each novelty is further classified by its impact on the agent's task performance:

- **Beneficial**: The novelty helps the agent complete the task more efficiently.
- **Detrimental**: The novelty hinders or prevents task completion unless the agent adapts.
- **Irrelevant**: The novelty does not affect the task and can be safely ignored.

Twelve novelties are implemented across all four tasks, spanning all combinations of class and type. Examples include a *fence* (object, detrimental) that obstructs access to resources, an *axe* (object and attribute, beneficial) that doubles the yield from breaking objects, *break increase* (attribute, beneficial) that provides additional items upon breaking, *fire wall* (object and attribute, detrimental) that causes episode termination upon contact, and *action remapping* (action, detrimental) that randomly shuffles the effects of the agent's actions. Some novelties span multiple classes; for instance, *axe-to-break* introduces a new object, adds a new action, and modifies the transition dynamics of the break action, spanning all three classes simultaneously.

3.2.3 Evaluation Metrics

NovelGridworlds introduces three evaluation metrics designed to capture different aspects of an agent's response to novelty:

- **Metric M1 (Reaction Performance)**: The ratio of cumulative action cost in the post-novelty scenario to the cost in the pre-novelty scenario. A ratio greater than 1 indicates degraded performance, less than 1 indicates improvement, and equal to 1 indicates no change.

- **Metric M2 (Steps)**: The number of steps the agent takes to complete the task in the post-novelty scenario.

- **Metric M3 (Detection Rate)**: A binary indicator of whether the agent correctly reports the presence of a novelty. This metric is specific to agents with explicit novelty detection mechanisms, such as planning agents.

3.2.4 Experimental Results

Two types of agents were evaluated: a symbolic planning agent implemented using the DIARC cognitive architecture [35, 36] with the MetricFF planner [21], and a reinforcement learning agent trained using Proximal Policy Optimization (PPO) [38]. The RL agent used a LiDAR-like observation space consisting of eight beams at 45° increments, each returning the Euclidean distance to the nearest object, augmented by inventory and selected-item vectors.

Planning agent. The planning agent was evaluated across all four tasks under three novelties (fence, arrow/spring, and axe). The agent achieved a 100% detection rate (M3) in all scenarios. Under the fence novelty, the cost ratio M1 consistently exceeded 1 across all tasks (for example, 1.80 ± 0.15 in Bow-v0 and 1.34 ± 0.06 in Pogostick-v1), reflecting the additional cost of breaking fences to access resources. Under irrelevant novelties (arrow/spring), the agent's performance remained close to the pre-novelty baseline, as expected. The axe novelty, although beneficial, produced negligible improvement because the agent did not always discover or exploit the new object.

Table 3.1: Performance of the planning agent across four tasks. Each task reports pre-novelty (Default) and post-novelty (Fence, Arrow/Spring, Axe) scenarios.

Metrics	Default	Fence	Arrow/Spring	Axe
NovelGridworld-Bow-v0				
M1	1.00 ± 0.00	1.80 ± 0.15	1.11 ± 0.135	1.02 ± 0.04
M2	87.6 ± 16.8	123.0 ± 18.0	90.6 ± 15.1	98.0 ± 15.9
NovelGridworld-Bow-v1				
M1	1.00 ± 0.00	1.82 ± 0.24	1.00 ± 0.02	0.99 ± 0.02
M2	36.8 ± 6.5	54.2 ± 4.8	37 ± 5.4	35.4 ± 4.0
NovelGridworld-Pogostick-v0				
M1	1.00 ± 0.00	1.16 ± 0.08	0.97 ± 0.03	0.99 ± 0.04
M2	127.8 ± 9.0	122.0 ± 19.8	105.6 ± 15.7	114.8 ± 4.0
NovelGridworld-Pogostick-v1				
M1	1.00 ± 0.00	1.34 ± 0.06	1.07 ± 0.05	0.99 ± 0.02
M2	94.2 ± 12.9	105.2 ± 14.5	100.8 ± 22.58	85.0 ± 15.4

Reinforcement learning agent. The RL agent was evaluated on the Bow-v0 task under three novelties: action remapping, fire wall, and break increase. The agent achieved a 92% success rate on the base task after 1.8 million training time steps. Under the beneficial break-increase novelty, cumulative reward improved but the success rate dropped to 29.2%, suggesting the agent learned to hoard resources rather than complete the task. The fire-wall novelty caused a complete failure, with 0% success rate, as the agent could not learn to avoid the lethal walls. Action remapping showed an average 12% decline in success rate, though performance varied substantially across trials depending on how severely the action mapping was shuffled.

3.2.5 Key Insights

The NovelGridworlds experiments reveal a fundamental tension between the two dominant paradigms for sequential decision-making. Planning agents, equipped with explicit models of the environment, excel at detecting novelties through discrepancies between expected and observed states. However, they lack mechanisms for adapting their models in response to detected novelties, and their performance degrades gracefully but without recovery. Reinforcement learning agents, in contrast, can potentially adapt through continued interaction but suffer from high sample complexity and are brittle when novelties alter the observation space or introduce lethal states. Neither paradigm alone is sufficient for robust open-world operation. These observations

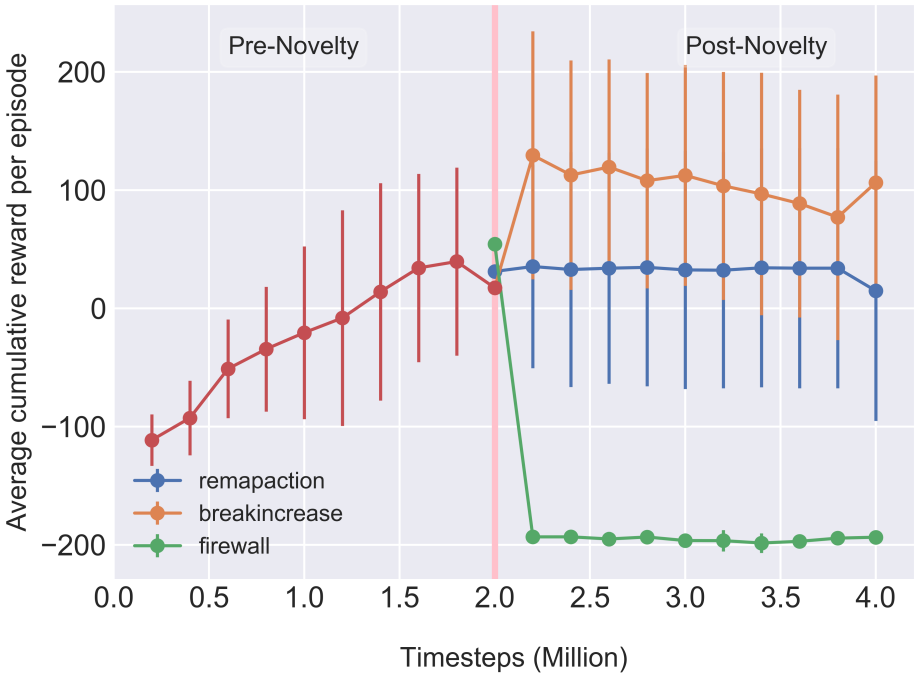


Figure 3.3: Learning performance of the RL agent on NovelGridworld-Bow-v0. The vertical line at 2 million time steps marks the introduction of novelty. Break increase (orange) shows improved reward but reduced success rate. Fire wall (green) causes complete failure. Action remapping (blue) shows partial recovery.

directly motivate the development of hybrid approaches, as well as a more flexible evaluation ecosystem, which we present in the next section.

3.3 NovelGym

NovelGym[2] extends and generalizes the ideas introduced in NovelGridworlds into a flexible ecosystem for developing and benchmarking open-world agents. While NovelGridworlds provides a fixed set of tasks and novelties suitable for initial evaluation, NovelGym introduces a formal framework for defining environments and their transformations, a modular software architecture for rapid development of tasks, novelties, and agent systems, and a comprehensive evaluation protocol with adaptation-focused metrics. The ecosystem supports single-agent and multi-agent scenarios, accommodates symbolic planning agents, reinforcement learning agents,

[2]The project website and codebase are available at `https://github.com/tufts-ai-robotics-group/NovelGym`. Detailed tutorials, PDDL specifications, configuration file formats, and hyperparameter settings are provided in Goel et al. [14].

Figure 3.4: NovelGym environment representation showing a gridworld with various entities. The red box highlights a novelty (axe) introduced in the environment.

and hybrid architectures, and enables composable novelty injection.

3.3.1 Environment Formalism

We formalize the environment as a tuple $E = \langle G, \mathcal{E}, \mathcal{R}, \mathcal{S}, \mathcal{A}, \tau, C \rangle$, where each component is defined as follows.

Grid. $G \subseteq \mathbb{Z}^2$ represents the set of all grid cells in a two-dimensional gridworld. For an $m \times n$ grid, each cell is identified by its position (i, j) with $1 \leq i \leq m$ and $1 \leq j \leq n$.

Entities. $\mathcal{E} = \{\langle t_1, P_{e_1} \rangle, \langle t_2, P_{e_2} \rangle, \ldots, \langle t_n, P_{e_n} \rangle\}$ is the set of all entities in the environment. Each entity $e = \langle t, P_e \rangle$ consists of a type t drawn from the set of all entity types T and a set of properties P_e drawn from a global property set P. Some entities are *dynamic*, meaning they can take actions in the environment and influence state transitions. Entities can be located on grid cells, in an agent's inventory, or nested within other entities.

Recipes. $\mathcal{R} = \{r_1, r_2, \ldots, r_r\}$ is a set of transformation rules. Each rule $r : \mathcal{E}^* \times L_r \to \mathcal{E}^*$ defines how one or more entities can be transformed into another set of entities, where $L_r \subseteq G$ restricts the locations at which the recipe can be applied. For example, a recipe might specify

that one unit of platinum, one stick, and two planks can be combined at the crafting table to produce a platinum axe.

States, actions, and transitions. $\mathcal{S}$ is the set of all possible states, capturing the locations and properties of all entities. $\mathcal{A} = \{a_1, a_2, \ldots, a_a\}$ is the set of primitive actions. While all actions are syntactically available in every state, some produce effects only in specific contexts. The transition dynamics are defined as:

$$\tau : \mathcal{S} \times \mathcal{A} \times (\mathcal{A}_{e_1}, \mathcal{A}_{e_2}, \ldots, \mathcal{A}_{e_m}) \to \mathcal{S},$$

where the tuple $(\mathcal{A}_{e_1}, \ldots, \mathcal{A}_{e_m})$ captures the actions taken by dynamic entities in the environment, reflecting the multi-agent nature of the system.

Cost function. $C : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^+$ assigns a non-negative cost to each state-action pair. Costs can depend on context; for example, breaking a tree may cost less when the agent holds a tool.

3.3.2 Agent Formalism

An agent is defined as $\mathcal{A} = \langle Sen, Act, \mathcal{K}, \Pi \rangle$, where Sen denotes the agent's sensors that generate observations O of the world, Act denotes actuators that can affect the world, $\mathcal{K}$ is a knowledge repository that may be initially empty and accumulate knowledge over time, and $\Pi : O \to \mathcal{A}$ maps observations to actions.

Observations can take multiple forms depending on the agent's sensor configuration. NovelGym supports LiDAR-like sensors that return distances to entities at angular increments, image-based local view representations using one-hot vector encodings of entity types on a grid around the agent, and symbolic state descriptions expressed in PDDL. This flexibility allows the same environment to interface with planning agents, reinforcement learning agents, and hybrid systems without modification.

The agent's behavior function Π can be implemented using symbolic planning, reinforcement learning, or hybrid approaches, as formalized in Sections 2.2.1 to 2.2.3. Actions can be primitive (e.g., `move-forward`) or parameterized operators (e.g., `approach <entity>`) that are executed through sequences of primitive actions by an internal planner.

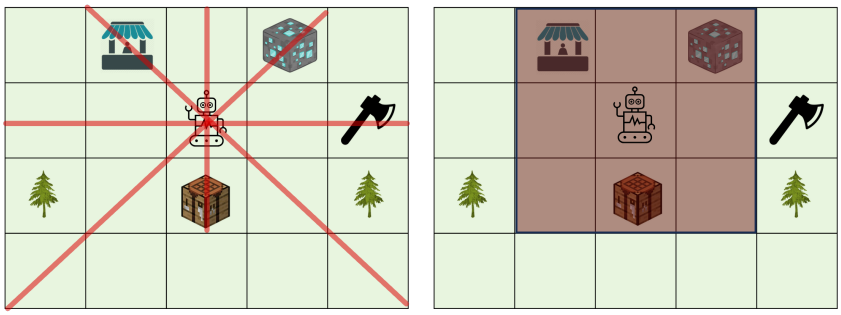


Figure 3.5: Illustration of the agent's sensor representations in NovelGym. (Left) A LiDAR-like representation emitting beams at angular increments. (Right) An image-based local view representation using one-hot vector encodings.

3.3.3 Novelty as Environment Transformation

NovelGym formalizes novelty through the concept of environment transformations. A transformation is a function $\nu : E \to E'$ that modifies one or more components of the environment. This formalism complements the class-based taxonomy introduced in Section 3.2.2: while the taxonomy categorizes novelties by which aspect of the agent's model they affect, the transformation formalism specifies precisely which components of the environment are modified and how. Six categories of transformation are supported:

- **Layout changes**: The grid G is modified such that $G \neq G'$, altering the spatial structure of the environment.
- **Entity alterations**: The set of entities $\mathcal{E}$ changes such that $\mathcal{E} \neq \mathcal{E}'$, introducing new entities or modifying existing ones.
- **Recipe modifications**: The transformation rules $\mathcal{R}$ change such that $\mathcal{R} \neq \mathcal{R}'$, altering how items can be crafted.
- **Action alterations**: The action set $\mathcal{A}$ changes such that $\mathcal{A} \neq \mathcal{A}'$, adding, removing, or modifying available actions.
- **Transition dynamics changes**: For some state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$, $\tau(s, a) \neq \tau'(s, a)$, changing how the environment responds to the agent's actions.
- **Cost function alterations**: The cost function C changes such that $C \neq C'$, modifying the cost associated with certain actions.

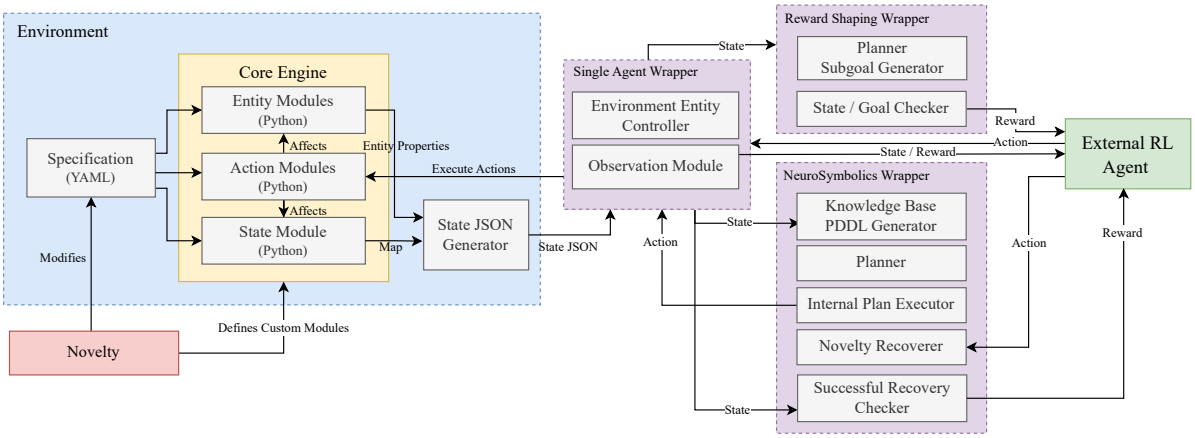


Figure 3.6: System design of the NovelGym ecosystem. Blue highlights the environment modules and purple highlights the agent modules. The novelty injector (red) transforms the environment based on specified configurations.

Critically, these transformations are *composable*. Multiple transformations can be applied in sequence to construct complex novelty scenarios, enabling the simulation of environments that undergo continual change. Whether a particular transformation constitutes a novelty for a given agent depends on the agent's knowledge, perceptions, and representations. An agent that navigates using LiDAR sensors may be unaffected by a layout transformation that changes the visual appearance of walls, whereas an agent relying on image-based observations would encounter novelty.

3.3.4 Architecture and Implementation

The NovelGym ecosystem is composed of two main components: the environment and the agent training architecture. A novelty injector module transforms the environment based on specified configurations.

Environment. The environment is implemented on top of PettingZoo [41] and OpenAI Gym [7], supporting multi-agent systems natively. It comprises several modules:

- A *specification module* that reads YAML configuration files and initializes the world, including entities, actions, and their properties.
- An *entity module* responsible for entity initialization and customization, including configurable properties such as breakability.

- An *action module* that implements primitive actions and higher-level operators and ensures the world state is updated upon execution.

- A *state module* that maintains the world map, tracks entities, and manages scheduled state updates for durative actions.

- A *novelty module* that modifies entity, action, and state modules according to the novelty configuration.

Agent training architecture. The agent-side architecture is designed to support hybrid planning and learning systems through three components:

- A *single-agent wrapper* that converts the multi-agent PettingZoo environment into a single-agent interface by controlling all non-primary agents. It includes an observation conversion module that can produce LiDAR, local-view, or symbolic state representations. For object-centric observation spaces, the wrapper automatically expands observation and action spaces when new entities or actions appear due to novelty.

- A *neurosymbolic wrapper* that maintains a PDDL-based knowledge base, automatically generates PDDL domain and problem files from the environment configuration, interfaces with a planner (MetricFF [21]) to generate plans, executes plan operators through a plan executor, and provides a novelty recovery component for implementing adaptation routines.

- A *reward-shaping wrapper* that uses the integrated planner to generate subgoals from the symbolic plan and shapes rewards by monitoring whether subgoals are achieved during learning. This wrapper enables sophisticated routines for novelty handling that combine planning and learning.

The architecture also integrates a modular reinforcement learning framework (Tianshou [43]), enabling users to experiment with various learning algorithms. The modularity of the system played a crucial role in enabling the implementation of complex hybrid agent architectures evaluated in this chapter and developed further in the RapidLearn line of work [13].

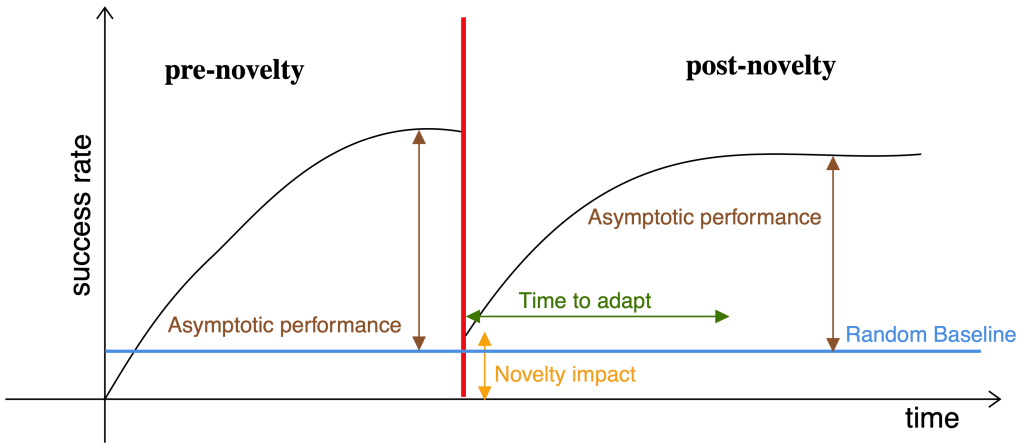


Figure 3.7: Illustration of the NovelGym evaluation protocol and performance over time: success rate as training progresses, with novelty injection dividing pre- and post-novelty regimes. The annotated regions correspond to the metrics in Section 3.3.5 (pre-novelty asymptotic performance, novelty impact, adaptation, and post-novelty performance).

3.3.5 Evaluation Protocol and Metrics

NovelGym introduces an evaluation protocol consisting of four phases, illustrated in Figure 3.7:

1. **Initial training**: The agent is trained in a controlled, novelty-free environment until its knowledge base or policy is sufficient to solve the task.

2. **Novelty injection**: A transformation is applied to the environment. The agent's performance is monitored immediately to assess the impact.

3. **Adaptation**: The agent interacts with the transformed environment to recalibrate its approach.

4. **Post-adaptation evaluation**: After a predefined adaptation period or upon meeting a convergence criterion, the agent's performance is assessed in the novel environment.

Five evaluation metrics are defined to capture different aspects of an agent's behavior under novelty:

1. **Pre-novelty asymptotic performance** (S_{pre}): The agent's success rate after convergence in the pre-novelty task.

2. **Novelty impact** (I_{novelty}): The immediate performance drop upon novelty introduction, defined as $I_{\text{novelty}} = S_{\text{pre}} - S_{\text{immediate post}}$.

3. **Time to adapt** (T_{adapt}): The number of time steps, actions, or wall-clock time required for the agent to reach a convergence criterion after novelty injection.

4. **Asymptotic adaptation performance** (S_{post}): The agent's success rate after adaptation has converged in the post-novelty scenario.

5. **Post-adaptation efficiency** (Δt): The difference in average task completion time before and after novelty, defined as $\Delta t = t_{\text{pre}} - t_{\text{post}}$. A positive value indicates that the novelty ultimately allowed the agent to find a more efficient solution.

These metrics extend the M1, M2, and M3 metrics of NovelGridworlds by explicitly capturing adaptation dynamics. While the earlier metrics focus on cost and detection, the NovelGym metrics additionally measure the speed of adaptation, the severity of the initial performance drop, and whether the agent can exploit beneficial novelties.

3.3.6 Benchmark Task and Novelties

The benchmark task in NovelGym is a Minecraft-inspired pogostick crafting task. The agent operates in a two-room gridworld environment containing resources (oak trees, diamond ore, platinum blocks), a crafting table, and a trader. The agent must collect resources, craft intermediary items, trade with the trader, and ultimately craft a pogostick. A simplified version of the task was used to make it tractable for reinforcement learning agents while preserving the sequential, multi-step nature of the crafting challenge.

Five environment transformations were selected for evaluation, spanning detrimental and beneficial categories:

1. **Axe** (detrimental): Trees become unbreakable unless the agent uses a newly introduced axe item.

2. **Chest** (beneficial): A chest containing all ingredients for crafting a pogostick is placed in the environment, along with a new action to approach it.

3. **Trader** (detrimental): The agent must maintain one grid cell of distance from the trader for the trade action to succeed.

Figure 3.8: NovelGym environment representation showing a gridworld with various entities. The red box highlights a novelty (axe) introduced in the environment.

4. **Fence** (detrimental): All oak logs are surrounded by breakable fences that must be removed before the logs can be accessed.

5. **Fire** (detrimental): The crafting table is set on fire, and the agent must collect a water bucket and extinguish the fire before crafting.

Twelve transformations are implemented in total, including additional novelties such as *busy traders*, *moving traders*, *multi-room layouts*, and *portal treasure*[3].

3.3.7 Agent Architectures

Four agent architectures were benchmarked, spanning hybrid and transfer learning approaches:

- **RapidLearn$^+$ (PPO+ICM)**: A hybrid planning and learning architecture that uses symbolic planning for the pre-novelty task and instantiates a reinforcement learning problem when plan execution fails due to novelty. The RL component uses PPO [38] augmented with the Intrinsic Curiosity Module (ICM) [30] for exploration. This architecture is a direct implementation of the method introduced in Goel et al. [13].

- **RapidLearn (PPO)**: The same hybrid architecture without ICM.

[3]A complete list of all implemented transformations and their specifications is provided in Goel et al. [14].

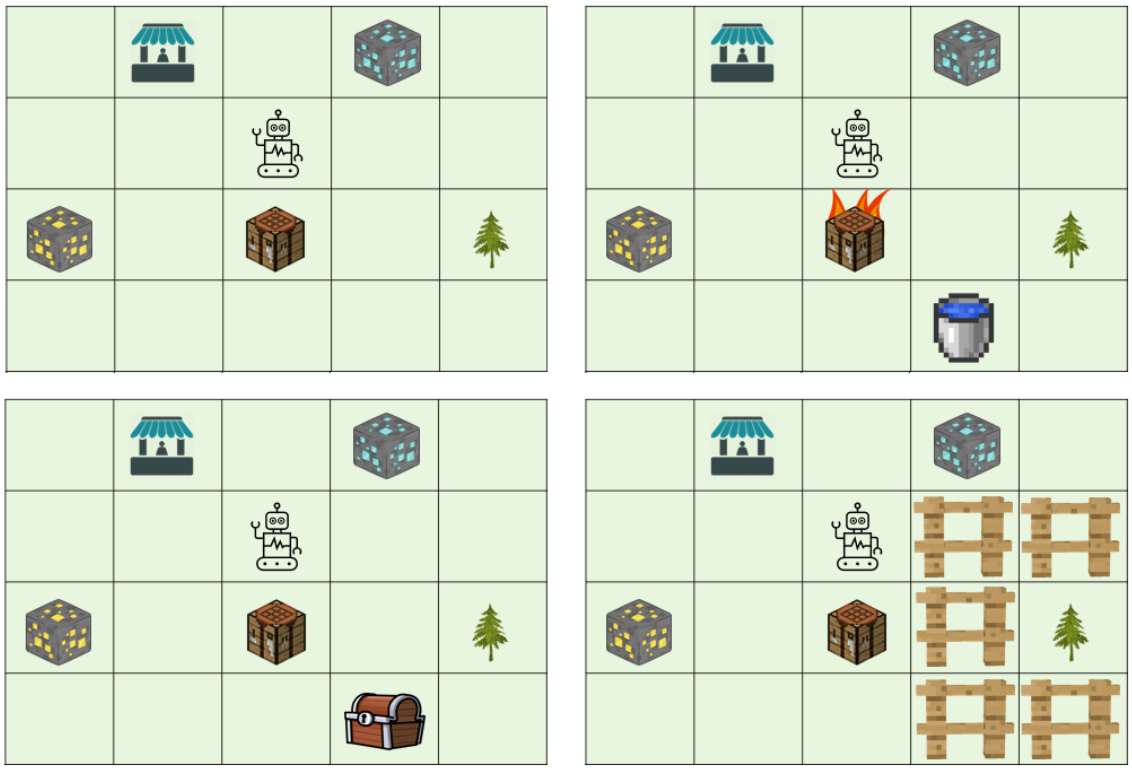


Figure 3.9: Illustration of the pre-novelty environment and three novelty scenarios: fire, fence, and chest. Each transformation modifies specific components of the environment formalism.

- **Transfer RL (PPO)+ICM**: A pure reinforcement learning approach that pre-trains on the base task and transfers weights to the post-novelty task, with automatic expansion of observation and action spaces, augmented with ICM.

- **Transfer RL (PPO)**: The same transfer approach without ICM.

All agents used a LiDAR-like observation space with beams emitted at $\frac{\pi}{4}$ increments for each entity type, producing an observation vector of size $8 \times |\mathcal{E}|$, augmented by inventory and selected-item sensors. The symbolic description for hybrid agents was represented in PDDL with a type hierarchy, predicates for entity relations, and operators defined with preconditions and effects.

Novelty	Agent	$S_{\text{pre}} \uparrow$	$I_{\text{nov}} \downarrow$	$S_{\text{post}} \uparrow$
Axe	RapidLearn$^+$(PPO+ICM)	1 ± 0	0.83 ± 0.205	1.0 ± 0.0
	RapidLearn (PPO)	1 ± 0	0.69 ± 0.207	1.0 ± 0.0
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.97 ± 0.021
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.97 ± 0.018
Chest	RapidLearn$^+$(PPO+ICM)	1 ± 0	–	–
	RapidLearn (PPO)	1 ± 0	–	–
	Transfer RL (PPO)+ICM	1 ± 0	0.0 ± 0.004	1.0 ± 0.0
	Transfer RL (PPO)	1 ± 0	0.01 ± 0.009	0.98 ± 0.011
Trader	RapidLearn$^+$(PPO+ICM)	1 ± 0	0.45 ± 0.069	0.95 ± 0.017
	RapidLearn (PPO)	1 ± 0	0.5 ± 0.07	0.96 ± 0.02
	Transfer RL (PPO)+ICM	1 ± 0	0.96 ± 0.108	0.99 ± 0.011
	Transfer RL (PPO)	1 ± 0	0.94 ± 0.128	0.96 ± 0.025
Fence	RapidLearn$^+$(PPO+ICM)	1 ± 0	0.38 ± 0.102	0.96 ± 0.02
	RapidLearn (PPO)	1 ± 0	0.41 ± 0.052	0.95 ± 0.008
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.93 ± 0.02
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.92 ± 0.024
Fire	RapidLearn$^+$(PPO+ICM)	1 ± 0	1.0 ± 0.0	0.95 ± 0.02
	RapidLearn (PPO)	1 ± 0	1.0 ± 0.0	0.95 ± 0.016
	Transfer RL (PPO)+ICM	1 ± 0	1.0 ± 0.0	0.94 ± 0.016
	Transfer RL (PPO)	1 ± 0	1.0 ± 0.0	0.96 ± 0.029

Table 3.2: Performance metrics across novelty scenarios.

3.3.8 Experimental Results

All agents achieved perfect pre-novelty performance ($S_{\text{pre}} = 1.0$), with the RL agents requiring 4 million training time steps to reach this level. Results across five novelties and four agents are summarized below; see Goel et al. [14] for the full quantitative tables[4].

Novelty impact. Upon novelty injection, all agents experienced performance degradation for detrimental novelties, with the fire novelty producing the most severe impact ($I_{\text{novelty}} = 1.0$ for all agents). Transfer RL agents were generally more sensitive to novelty than hybrid agents, as reflected in higher I_{novelty} values. However, hybrid agents showed lower immediate impact in some cases due to their targeted adaptation approach, which activates only upon execution failure.

Adaptation speed. Hybrid agents consistently outperformed transfer learning methods on the T_{adapt} metric across all novelties. For the fence novelty, hybrid agents adapted in approxi-

[4] All results are averaged across 10 random seeds. A convergence criterion based on success rate and reward thresholds was used to determine when adaptation was complete; see Goel et al. [14] for the precise convergence criteria and hyperparameter settings.

Novelty	Agent	$T_{\text{adapt}} \downarrow$	$\Delta t \downarrow$
Axe	RapidLearn$^+$(PPO+ICM)	71040 ± 17413	122.7 ± 15.61
	RapidLearn (PPO)	68160 ± 17934	116.9 ± 9.88
	Transfer RL (PPO)+ICM	169440 ± 97949	61.0 ± 8.95
	Transfer RL (PPO)	114240 ± 19651	63.1 ± 8.17
Chest	RapidLearn$^+$(PPO+ICM)	–	–
	RapidLearn (PPO)	–	–
	Transfer RL (PPO)+ICM	24000 ± 0	-3.0 ± 0.78
	Transfer RL (PPO)	24000 ± 0	-3.5 ± 3.13
Trader	RapidLearn$^+$(PPO+ICM)	102720 ± 20724	126.9 ± 9.97
	RapidLearn (PPO)	96480 ± 21191	122.8 ± 8.93
	Transfer RL (PPO)+ICM	227040 ± 112120	93.6 ± 12.05
	Transfer RL (PPO)	350880 ± 237039	89.3 ± 7.87
Fence	RapidLearn$^+$(PPO+ICM)	69120 ± 15657	161.6 ± 7.6
	RapidLearn (PPO)	66400 ± 20999	168.7 ± 6.15
	Transfer RL (PPO)+ICM	854400 ± 52974	73.4 ± 4.7
	Transfer RL (PPO)	701760 ± 100589	90.9 ± 9.49
Fire	RapidLearn$^+$(PPO+ICM)	263520 ± 81979	133.2 ± 21.38
	RapidLearn (PPO)	252000 ± 49663	142.5 ± 13.71
	Transfer RL (PPO)+ICM	372960 ± 198096	105.3 ± 9.85
	Transfer RL (PPO)	127200 ± 37932	103.3 ± 13.22

Table 3.3: Efficiency metrics across novelty scenarios.

mately 67,000–71,000 time steps, while transfer RL agents required 700,000–854,000 time steps. This advantage arises from the hybrid architecture's ability to focus learning on the specific subproblem caused by the novelty rather than relearning the entire task.

Post-adaptation performance. All agents achieved high post-adaptation success rates ($S_{\text{post}} \geq 0.92$), though none reached perfect scores due to the use of a convergence criterion rather than exhaustive training.

Beneficial novelty. The chest novelty was not detected as a novelty by hybrid agents, since it did not cause execution failure. Transfer RL agents, however, discovered and exploited the chest through exploration, as reflected in negative Δt values indicating improved task completion efficiency.

Role of curiosity-driven exploration. The addition of ICM improved adaptation in some cases but was not uniformly beneficial. In the fire novelty scenario, ICM slightly degraded the performance of the transfer RL agent, suggesting that intrinsic curiosity can sometimes interfere with goal-directed adaptation.

Bibliography

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] Jonathan C Balloch, Zhiyu Lin, Robert Wright, Xiangyu Peng, and Mark O Riedl. NovGrid: A flexible grid world environment for studying novelty. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 2022.

[3] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.

[4] Abhijit Bendale and Terrance E Boult. Towards open set deep networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1563–1572, 2016.

[5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A Vision-Language-Action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[6] Terrance E Boult, Henry Grabowecky, Daniel Prijatel, Roberto Fernandez, Lawrence Holder, and Joshua Alspector. Towards a unifying framework for formal theories of novelty. *arXiv preprint arXiv:2103.02529*, 2021.

[7] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[8] Richard E Fikes and Nils J Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4):189–208, 1971.

[9] Maria Fox, Alfonso Gerevini, Derek Long, Ivan Serina, et al. Plan stability: Replanning versus plan repair. In *ICAPs*, volume 6, pages 212–221, 2006.

[10] Christian Fritz and Sheila A McIlraith. Monitoring plan optimality during execution. In *ICAPS*, pages 144–151, 2007.

[11] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4: 265–293, 2021.

[12] Shivam Goel, Gyan Tatiya, Matthias Scheutz, and Jivko Sinapov. Novelgridworlds: A benchmark environment for detecting and adapting to novelties in open worlds.

[13] Shivam Goel, Yash Shukla, Vasanth Sarathy, Matthias Scheutz, and Jivko Sinapov. Rapid-learn: A framework for learning to recover for handling novelties in open-world environments. In *IEEE International Conference on Development and Learning (ICDL), London, UK, September 12-15, 2022*, pages 1–8. IEEE, 2022. URL `https://arxiv.org/pdf/2206.12493`.

[14] Shivam Goel, Yichen Wei, Panagiotis Lymperopoulos, Klára Churá, Matthias Scheutz, and Jivko Sinapov. Novelgym: A flexible ecosystem for hybrid planning and learning agents designed for open worlds. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pages 688–696, 2024.

[15] Xiuye Gu, Tsung-Yi Lin, Weicheng Kuo, and Yin Cui. Open-vocabulary object detection via vision and language knowledge distillation. In *International Conference on Learning Representations (ICLR)*, 2022.

[16] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[17] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.

[18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[19] Malte Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

[20] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[21] Jörg Hoffmann. The Metric-FF planning system: Translating "ignoring delete lists" to numeric state variables. *Journal of Artificial Intelligence Research*, 20:157–194, 2003.

[22] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.

[23] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Hierarchical task and motion planning in the now. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1470–1477, 2011.

[24] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.

[25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska,

et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[26] Pat Langley. Open-world learning for radically autonomous agents. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(09):13539–13543, 2020.

[27] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. *Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control*, 1998.

[28] Matthias Minderer, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, Aravindh Mahendran, Anurag Arnab, Mostafa Dehghani, Zhuoran Shen, et al. Simple open-vocabulary object detection. In *European conference on computer vision*, pages 728–755. Springer, 2022.

[29] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[30] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning (ICML)*, pages 2778–2787, 2017.

[31] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018.

[32] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[33] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[34] Walter J Scheirer, Anderson de Rezende Rocha, Archana Sapkota, and Terrance E Boult. Toward open set recognition. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 35, pages 1757–1772. IEEE, 2013.

[35] Paul Schermerhorn, James Kramer, Timothy Brick, David Anderson, Aaron Dingler, and Matthias Scheutz. DIARC: A testbed for natural human-robot interaction. In *AAAI Workshop on Mobile Robot Competition and Exhibition*, 2006.

[36] Matthias Scheutz, Selmer Bringsjord, and Naveen Sundar Govindarajulu. An overview of the distributed integrated cognition affect and reflection DIARC architecture. In *Cognitive Architecture Integrations: Common Model of Cognition and Beyond*, pages 201–224. Springer, 2019.

[37] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588 (7839):604–609, 2020.

[38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017.

[39] Ted Senator. Science of artificial intelligence and learning for open-world novelty (SAIL-ON). *DARPA Broad Agency Announcement HR001119S0038*, 2019.

[40] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[41] Jordan K. Terry, Benjamin Black, Nathaniel Grammel, Ananth Hari Jayakumar, Sanghyun Lee, Luis Santos, Claus Dieffendahl, Caroline Horsch, and Rodrigo Pérez-Vicente. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[42] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.

[43] Jiayi Weng, Min Chen, Yujie Shi, Minghao Li, Zhihan Zhang, Feiyang Li, Jun Zhu, and Yun Liu. Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research*, 23(267):1–6, 2022.

[44] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL `https://proceedings.mlr.press/v229/zitkovich23a.html`.